Getting Set Up

Local tools, Jetstream2 connection, Jupyter crashcourse

Quick Check — What Do You Have?

Raise your hand (or type in chat):

- **Do you have VS Code installed?**
- **Do you have Python installed?** (any version)
- **Can you open a terminal?** (PowerShell, Terminal, Git Bash, CMD)

If you answered **no** to any of these — no worries. The next two slides walk through getting everything installed. We'll have a **breakout room with helpers** to get everyone set up.

If you already have all three, you can skip ahead or help a neighbor.

Install VS Code + Extensions

1. Download VS Code

code.visualstudio.com

- Windows, macOS, or Linux
- Free, lightweight, widely used in research
- Run the installer, accept defaults

2. Install Extensions

Open VS Code → click the **Extensions** icon (left sidebar) → search and install:

1. **Python** (by Microsoft)
2. **Jupyter** (by Microsoft)

That's it — VS Code can now open `.ipynb` notebooks and run Python code.

Already have VS Code? Just make sure the Python and Jupyter extensions are installed.

Install Python

Option A: python.org

python.org/downloads

1. Download Python 3.12 (or latest 3.x)
2. **Windows:** check **"Add python.exe to PATH"** during install
3. macOS / Linux: usually pre-installed; installer if not

Option B: Anaconda / Miniconda

docs.anaconda.com/miniconda

- Comes with `conda` for environment management
- Good if you plan to use multiple Python versions
- Miniconda is the lightweight version

Verify it works — open a terminal and type:

```
python --version       # should print Python 3.x.x
pip --version          # should print pip 2x.x
```

If `python` doesn't work, try `python3`. On Windows, you may need to restart your terminal after installing.

Open a Terminal

You'll use a terminal to SSH into Jetstream2. Here's how to open one:

OS	Terminal App	How to Open
Windows	PowerShell or Windows Terminal	Start menu → type `powershell` or `terminal`
macOS	Terminal	Spotlight (Cmd+Space) → type `terminal`
Linux	Terminal / Konsole / GNOME Terminal	Ctrl+Alt+T (most distros)
VS Code	Integrated terminal	Ctrl+` (backtick) — works on all platforms

Test it: Type `ssh` and press Enter. You should see usage info, not "command not found."

Windows note: *If* `ssh` *is not found, install Git for Windows — it includes Git Bash with SSH built in.*

Optional: Generating an SSH Key Pair

Some allocations require an SSH key to create instances. If Exosphere prompts you for one, follow these steps. Otherwise, you can skip this — Exosphere will auto-generate a passphrase for you.

OS	Command
macOS / Linux	`ssh-keygen -t ed25519 -C "your_email@example.com"`
Windows	Open PowerShell: `ssh-keygen -t ed25519 -C "your_email@example.com"`

Step-by-step:

1. Run the command above — press **Enter** to accept the default file location
2. Enter a passphrase (optional but recommended)
3. Two files are created: `~/.ssh/id_ed25519` (private — never share) and `~/.ssh/id_ed25519.pub` (public — this goes to Exosphere)
4. Copy your public key: `cat ~/.ssh/id_ed25519.pub`
   On Windows PowerShell: `Get-Content ~\.ssh\id_ed25519.pub`
5. Paste it into Exosphere when creating your instance

Already have an SSH key? You can reuse it — just paste the public key (`.pub` file) into Exosphere.

Setup Breakout Room

If you need help installing VS Code, Python, or getting your terminal working — **join the Setup Breakout Room now.**

Helpers are standing by to walk you through it.

Once you have:

- VS Code with Python + Jupyter extensions
- Python 3.x working in your terminal (`python --version`)
- SSH available (`ssh` shows usage info)

… you're ready. Come back to the main room and we'll connect to Jetstream2.

If everything is already working, sit tight — we'll continue in a few minutes.

Scenario A: Your Own Instance

Create and configure a Jetstream2 VM from scratch

A1: Create a Jetstream2 Instance

1. Go to jetstream2.exosphere.app — log in with your ACCESS credentials
2. Select allocation **NAIRR250048**
3. **Create > Instance > Ubuntu 24.04**
4. Pick your flavor:

Flavor	CPUs	RAM	GPU	Good For
m3.medium	8	30 GB	—	CPU-only work, most research
g3.small	8	30 GB	A100 (40 GB)	**Today's exercises (recommended)**
g3.xl	32	120 GB	A100 (40 GB)	Larger GPU workloads

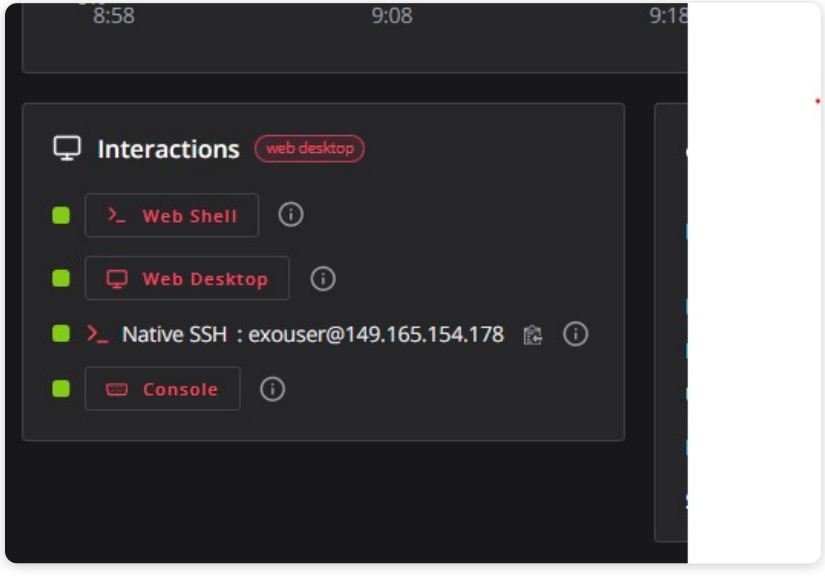
5. Enable **Web Desktop** > click **Create**
6. Wait ~3.5 minutes for "Ready"

A2: Find Your Connection Details

Click on your instance name in Exosphere. You need two things:

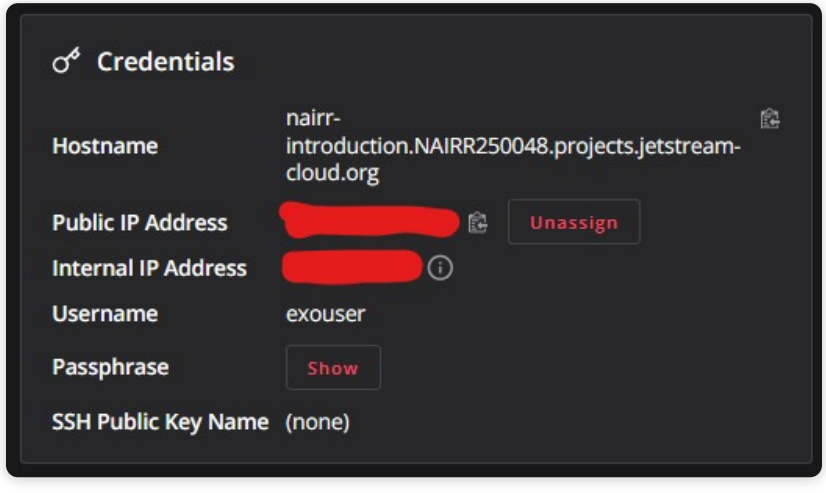
1. SSH Address

Under **Interactions**, find **Native SSH** — it shows `exouser@<ip>`. Click the copy icon.



2. Passphrase

Under **Credentials**, click **Show** next to Passphrase. This is your SSH password — keep it private.



Exosphere auto-generates a passphrase for each instance. You'll type it when SSH prompts for a password.

A3: Connect via SSH Tunnel

In your terminal:

```
ssh -L 8888:localhost:8888 exouser@<your-instance-ip>
```

Enter the passphrase when prompted. You're now on the Jetstream2 instance.

What this does:

- Connects you to the remote machine via SSH
- The `-L 8888:localhost:8888` part creates a **tunnel** — port 8888 on your laptop forwards to port 8888 on the instance
- This is how JupyterLab (running remotely) will appear in your local browser

A4: Set Up the Python Environment

First, clone the workshop materials:

```
git clone https://github.com/MattyTheBoi/NAIRR_Intro.git
cd NAIRR_Intro
```

The instance has system Python but no conda or JupyterLab. Jetstream2 provides conda via **modules**:

```
module load miniforge                       # makes conda available
conda create -n workshop python=3.12 -y     # create an isolated environment
conda init && source ~/.bashrc              # one-time shell setup
conda activate workshop                     # switch into the environment
pip install jupyterlab ipykernel            # install Jupyter + kernel support
pip install -r requirements.txt             # install exercise dependencies
```

Register the kernel so JupyterLab can find it:

```
python -m ipykernel install --user --name workshop \
    --display-name "Workshop (Python 3.12)"
```

A5: Launch JupyterLab

On the instance (inside your SSH session):

```
conda activate workshop
jupyter lab --no-browser --port=8888
```

You'll see output like:

```
http://localhost:8888/lab?token=abc123...
```

On your local machine, open that full URL (with token) in your browser.

Because of the SSH tunnel, `localhost:8888` on your machine connects to port 8888 on the Jetstream2 instance. JupyterLab is running *there*, but you see it *here*.

Copy the full URL including the token — JupyterLab needs it the first time you connect.

Scenario B: Shared Instance

The instructor set up an account for you — just connect

B1: Your Credentials

You have your own **Linux account** on a shared Jetstream2 instance. The instructor will share your credentials:

Field	Example
Username	`user07`
Password	`nairr2026`
Port	`8894`
Instance IP	`149.165.152.105`

You have your own home directory, your own JupyterLab process, and all 4 notebooks pre-loaded. Other participants cannot see your files.

Check the chat or spreadsheet the instructor shared for your specific username and port number.

B2: Connect & Open JupyterLab

Step 1: Open your terminal and SSH in:

```
ssh -L 8894:localhost:8894 user07@149.165.152.105
```

Replace `user07`, `8894`, and the IP with your values. Enter the password when prompted.

Step 2: Open your browser to:

```
http://localhost:8894/lab
```

No token needed — JupyterLab opens directly to your home directory with all notebooks ready.

If JupyterLab doesn't load: In your SSH session, run:

```
bash ~/start_jupyter.sh
```

Then refresh the browser. This restarts your JupyterLab process on your assigned port.

It’s Just a Machine

Your Jetstream2 instance is a **full Ubuntu 24.04 Linux machine**. You have root access (Scenario A) or your own user account (Scenario B). You can:

- `ssh` in and use the terminal directly
- Install any software: `apt install`, `pip install`, `conda install`
- Run Python scripts, shell scripts, Docker containers — anything
- Use the **Web Desktop** button in Exosphere for a full graphical Linux desktop

Today we use JupyterLab because it’s ideal for interactive, exploratory work. But this machine is yours to use however fits your research.

For research labs: *a single instance can serve your entire group — each researcher gets their own Linux account, they share data and the same environment. That’s exactly how the shared instance works today.*

Python Environments — Why They Matter

A **Python environment** is an isolated set of packages and a Python version. Think of it as a self-contained workspace for a project.

Why use them?

- **No conflicts** — Project A needs PyTorch 2.1, Project B needs 2.4? No problem.
- **Reproducible** — share `requirements.txt` and anyone can recreate your setup
- **Clean** — delete the env when you're done, system Python is untouched

Creating one

```
conda create -n myproject python=3.12
conda activate myproject
pip install numpy pandas torch
```

Today we use `conda activate workshop` — a pre-built environment with all exercise dependencies.

If you're used to R or MATLAB, this is new. In Python it's standard practice — every project gets its own environment.

Kernels — Where Your Code Runs

A **kernel** is the Python process that executes your notebook code. When you pick a kernel, you're choosing *which Python environment* runs the cells.

In JupyterLab

- Kernel name appears in the **top-right** of the notebook
- Click it to switch to a different environment
- If it says `workshop`, your code runs in the `workshop` conda env

In VS Code

- **"Select Kernel"** button in the top-right
- Lists all detected Python environments (conda, venv, system)
- Pick `workshop` for Jetstream2, or your local env for laptop runs

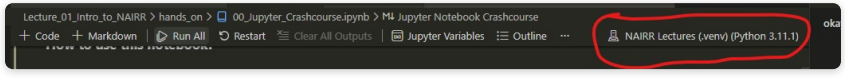
Key idea: Same notebook, different kernel = code runs in a different place with different packages. This is how you switch between local and Jetstream2.

The crashcourse notebook includes a cell that shows which kernel/environment you're using.

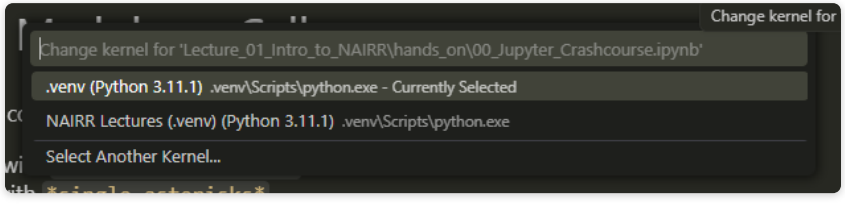
Advanced: VS Code → Jetstream2 (Remote Kernel)

You can also use **VS Code on your laptop** to edit notebooks while **code runs on Jetstream2**. Same SSH tunnel, familiar editor. For today's exercises, we use the **browser for Jetstream2** and **VS Code for local**.

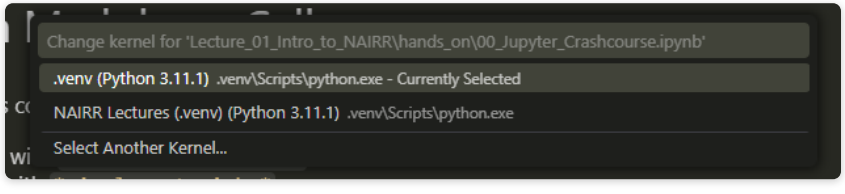
1. See the active kernel (top-right) — click it



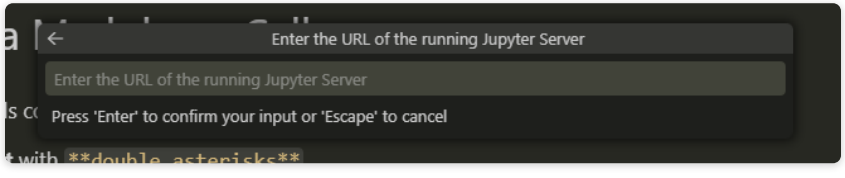
2. Click **"Select Another Kernel..."**



3. Choose **"Existing Jupyter Server..."**



4. Paste `localhost:<port>` — 8888 (Scenario A) or your assigned port (Scenario B)



Then pick Workshop (Python 3.12) — now every cell runs on Jetstream2. Works for both Scenario A (`exouser`) and Scenario B (`userNN`). Swap back to a local kernel anytime.

Two Environments, One Notebook

Each exercise has a **timing comparison table** with two columns. You'll run the same notebook in two places:

Where	How	Records
Browser	Open `http://localhost:<port>/lab` — JupyterLab on Jetstream2	Jetstream2 column
VS Code	Open the same `.ipynb` file locally — select your **local Python kernel**	Local (your laptop) column

Setup for local runs:

1. Clone the workshop repo:

```
git clone https://github.com/MattyTheBoi/NAIRR_Intro.git
cd NAIRR_Intro
```

2. Create a virtual environment and activate it:

```
python -m venv nairr-workshop
```

   Windows: `nairr-workshop\Scripts\activate` | **macOS/Linux:** `source nairr-workshop/bin/activate`

3. Install dependencies: `pip install -r requirements.txt`
4. In VS Code → **Select Kernel** → pick **nairr-workshop** (your local venv)

We'll do the Jetstream2 runs together first. You can do the local runs during the session or afterward.

Jupyter Notebook Crashcourse

The tool you'll use for every exercise

What Is a Jupyter Notebook?

- A **browser-based interactive environment** for writing and running code
- Mix **code, formatted text, and output** (including plots) in one document
- Python-based, but also supports R, Julia, and other languages
- File format: `.ipynb` — a JSON file that captures code, output, and narrative together

The name: Julia + **Py**thon + **R** — but today we use Python exclusively.

Jupyter is the standard tool for reproducible computational research. If you've used MATLAB's Live Editor or Mathematica notebooks, the concept is similar.

Why Researchers Use Jupyter

For your own work:

- Inline visualizations — plots appear right below the code
- Iterative exploration — run one cell at a time, inspect results, adjust
- Mix narrative and code — document your reasoning alongside the analysis
- Reproducible — anyone can re-run your notebook and get the same results

For collaboration:

- Share `.ipynb` files with colleagues — they see code + results
- GitHub renders notebooks in the browser
- Export to PDF or HTML for reports
- Common in publications as supplementary material

This is not just a teaching tool. Researchers at CERN, NASA, and major tech companies use Jupyter notebooks in production research workflows.

Where You Can Run Jupyter

Option	How	When to Use
VS Code	Install the Jupyter extension, open `.ipynb` files	Day-to-day local work, integrated with your editor
JupyterLab (browser)	`pip install jupyterlab`, then `jupyter lab`	Standalone environment, full-featured interface
Google Colab	Open colab.research.google.com	Quick experiments, free GPU, no setup
Jetstream2 / NAIRR	SSH tunnel + `jupyter lab` on the instance	Large-scale compute, persistent environment

Right now: you're using JupyterLab on Jetstream2. All four options run the same `.ipynb` files — your notebook is portable.

The JupyterLab Interface

Layout:

- **File browser** on the left — navigate, upload, create files
- **Notebook area** in the center — your cells live here
- **Toolbar** at the top — run, stop, restart, cell type dropdown
- **Kernel indicator** top-right — circle is filled when busy

Two cell types:

- **Code cells** — write Python, press `Shift+Enter` to run, output appears below
- **Markdown cells** — formatted text (headers, bold, links, lists). Double-click to edit, `Shift+Enter` to render

Key shortcuts:

- `Shift+Enter` — run cell, move to next
- `Ctrl+Enter` — run cell, stay in place

Open the Crashcourse Notebook

Notebook: `00_Jupyter_Crashcourse.ipynb`

Open it in JupyterLab and follow along. We'll work through it together, flipping back to these slides as needed.

What you'll practice:

- Running code cells and editing markdown cells
- Python basics: variables, lists, functions, imports
- Installing packages with `!pip install`
- Working with DataFrames and inline plots
- Keyboard shortcuts that speed up your workflow
- Restart & Run All for reproducibility

~15–20 minutes. The last cell verifies your Jetstream2 connection.

Verify Your Connection

The last cell of the crashcourse notebook runs a quick check:

```
import platform, os
print(f"Hostname:  {platform.node()}")
print(f"CPU cores: {os.cpu_count()}")
print(f"OS:        {platform.platform()}")
!nvidia-smi 2>/dev/null || echo "CPU instance (no GPU)"
```

What to look for:

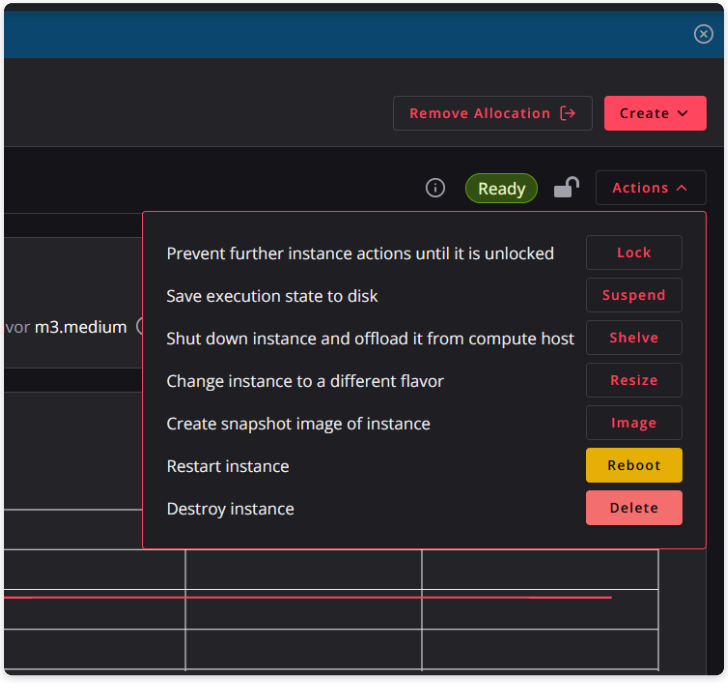
- **Hostname** — should be your Jetstream2 instance name, not your laptop
- **CPU cores** — 8 for m3.medium (vs. likely 4–8 on your laptop)
- **GPU** — if on a g3 instance, `nvidia-smi` will show the A100/H100

Everyone seeing their instance? Great — let's move to the exercises.

Before You Go: Shelve Your Instance

Scenario A only — if you created your own Jetstream2 instance, **shelve it** when you're done for the day. This frees compute resources for other researchers until our next session.

1. Go to your Exosphere dashboard
2. Click the **Actions** menu on your instance
3. Select **"Shelve"**



This shuts down the instance and offloads it from the compute host. Your files and configuration are preserved — just **unshelve** before the next session and everything will be exactly as you left it.

Scenario B participants: nothing to do — the instructor manages the shared instance.